

Query Optimization

* The process of providing equivalent query but with better strategy.

* indexed using id

..... WHERE bday > 2005 AND id > 5;

* Less optimized

..... WHERE id > 5 AND bday > 2005;

Optimized

* Extensive optimization is done for frequently used queries

* DB doesn't always choose the absolute fastest one as query optimization is costly as well.

Physical Optimization:

After getting query tree:

- ✓ 1. Efficient algorithm
- ✓ 2. Best execution order

Principle:

1. Reduce Disk I/O.
2. Reduce memory usage / Buffer consumption.
3. Filter data early.

Optimization happens on query tree based on rules:

v.9 1. Heuristic Rule-Based Algebraic Optimization [Rule]

2. Cost based optimization

2. Cost Based Optimization

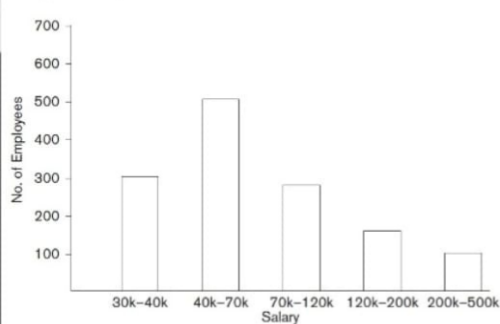
Table_name	Column_name	Num_distinct	Low_value	High_value
PROJECT	Plocation	200	1	200
PROJECT	Pnumber	2000	1	2000
PROJECT	Dnum	50	1	50
DEPARTMENT	Dnumber	50	1	50
DEPARTMENT	Mgr_ssn	50	1	50
EMPLOYEE	Ssn	10000	1	10000
EMPLOYEE	Dno	50	1	50
EMPLOYEE	Salary	500	1	500

Table_name	Num_rows	Blocks
PROJECT	2000	100
DEPARTMENT	50	5
EMPLOYEE	10000	2000

Index_name	Uniqueness	Blevel*	Leaf_blocks	Distinct_keys
PROJ_PLOC	NONUNIQUE	1	4	200
EMP_SSN	UNIQUE	1	50	10000
EMP_SAL	NONUNIQUE	1	50	500

*Blevel is the number of levels without the leaf level.

Histogram of salary in the relation EMPLOYEE.



↳ stores attributes

↳ Predicts how many rows are selected.

Database Catalog :

Table_name	Column_name	Num_distinct	Low_value	High_value
PROJECT	Plocation	200	1	200
PROJECT	Pnumber	2000	1	2000
PROJECT	Dnum	50	1	50
DEPARTMENT	Dnumber	50	1	50
DEPARTMENT	Mgr_ssn	50	1	50
EMPLOYEE	Ssn	10000	1	10000
EMPLOYEE	Dno	50	1	50
EMPLOYEE	Salary	500	1	500

Table_name	Num_rows	Blocks
PROJECT	2000	100
DEPARTMENT	50	5
EMPLOYEE	10000	2000

Index_name	Uniqueness	Blevel*	Leaf_blocks	Distinct_keys
PROJ_PLOC	NONUNIQUE	1	4	200
EMP_SSN	UNIQUE	1	50	10000
EMP_SAL	NONUNIQUE	1	50	500

*Blevel is the number of levels without the leaf level.

① Retrieve / Join early : Storage

Smaller blocks/row → GET

[Down the RA tree]

A(5) B(15) C(10)



② Pipeline :

Smaller blocks/row → Buffered

Larger blocks/row → Pipelined



③ Selection :

Linear search : Not in index table

Index scan : In index table + Non unique 1

Index seek : In index table + Unique

④ Join :

Nested Loop Join : Check blocks [Lesser one in the outer loop]

Merge Join : Both tables are indexed

Hash Join : Less [Hashed] + More [Pipeline]
Block Block

Index Join : Both are indexed based on the column

Question:

① Consider two tables:

- Table A (Student) — 500 tuples
- Table B (Enrollment) — 50,000 tuples

Given the SQL query:

Sql

```
SELECT A.Name, B.Course_id
FROM Student A, Enrollment B
WHERE A.Student_id = B.Student_id
AND A.Dept = 'CSE'
ORDER BY B.Course_id;
```

Answer the following:

(a) Identify each operation in this query as **pipelined** or **pipeline-breaking** (materialized). Justify briefly.

(b) Which of the three join algorithms — **Nested Loop Join**, **Merge Join**, or **Hash Join** — would be most efficient for `A.Student_id = B.Student_id`, given the size difference between A and B? Explain why.

(c) Suppose instead both tables are of similar size and already sorted on `Student_id`. Would your answer to (b) change? Explain.

A (500) B (50 000)

A.Dept = 'CSE'

Hash Join ; B (50,000) - Pipeline

A (500) - Hash bucket

Merge Join

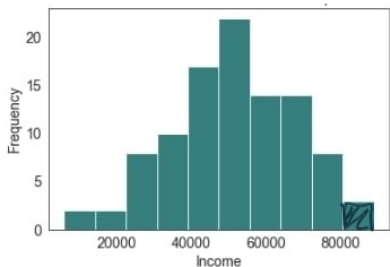
2

Table_name	Column_name	Num_distinct	Low_value	High_value
PROJECT	Plocation	200	1	200
PROJECT	Pnumber	2000	1	2000
PROJECT	Dnum	50	1	50
DEPARTMENT	Dnumber	50	1	50
DEPARTMENT	Mgr_ssn	50	1	50
EMPLOYEE	Ssn	10000	1	10000
EMPLOYEE	Dno	50	1	50
EMPLOYEE	Salary	500	1	100000

Table_name	Num_rows	Blocks
PROJECT	2000	100
DEPARTMENT	50	5
EMPLOYEE	10000	2000

(a) From the database catalogue information above, decide if a query join Project, Employee, and Departments; in general, which relation's records should be retrieved and joined earlier and why.

(b) In addition to the catalogue if a histogram on Employee relation's salary attribute is as follows



Then in the following query about Employees, which selection is more critical and which relation's records should be retrieved first for checking the join condition for best performance?

```
Select Fname, Lname
from EMPLOYEE join PROJECT
on Essn = Ssn
where salary > 80000 and Pnumber > 1000
```

Justify your answer.

Department

Employee

3 You are managing a database for a large e-commerce platform. You have two primary tables: Users (1 million records) and Orders (50 million records). Users frequently run a query to find their own order history:

```
SELECT * FROM Orders WHERE user_id = ? ORDER BY order_date DESC;
```

The database is currently performing a full table scan on the Orders table, which is causing significant latency.

- **Question:** As a database administrator, what two specific optimization strategies would you implement to improve the performance of this query? Justify your choices using the principles of Query Optimization (e.g., Disk I/O reduction, Indexing).

1 Selection Pushing
2 Composite index

4 You have a complex reporting query that joins three tables: Sales, Products, and Regions.

- Sales: 10,000,000 rows
- Products: 5,000 rows
- Regions: 50 rows
- The query includes filters for Region = 'North America' and Product_Category = 'Electronics'.
- **Question:** Explain how a Cost-Based Optimizer would prioritize the join order. Which table would it likely join first, and why? Explain the concept of "Selection Pushing" in the context of this specific scenario to minimize intermediate results.

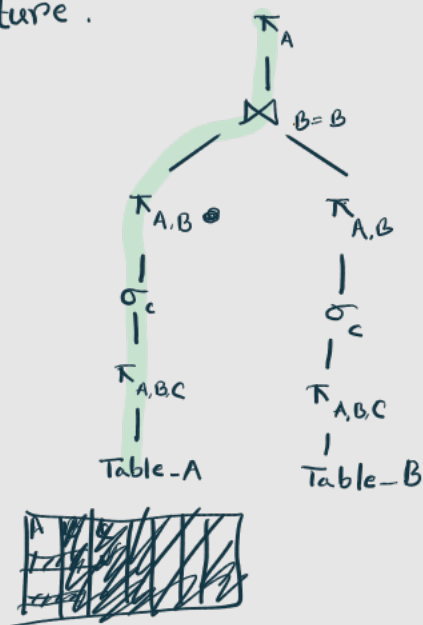


First join: R and P
Then join S.

1 Heuristic Rule Based Algebraic Optimization

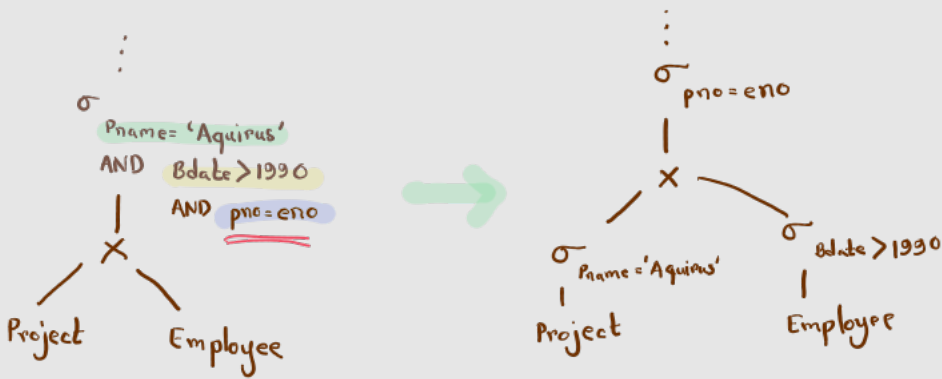
Structure :

selection pushing



① Heuristic Rule Based Algebraic Optimization

(1) Break down selection (σ_c):



② Interchange

A(100) B(10) C(5)



③ $\sigma_c + X = \Join_c$



Algebraic Modification Rules

Rule	σ — Selection	π — Projection
Cascade	$\sigma_{C_1 \text{ AND } C_2}(R) = \sigma_{C_1}(\sigma_{C_2}(R))$	$\pi_A(\pi_B(R)) = \pi_A(R)$
Commutative	$\sigma_{C_1}(\sigma_{C_2}(R)) = \sigma_{C_2}(\sigma_{C_1}(R))$ ✓	$\pi_A(\pi_B(R)) \neq \pi_B(\pi_A(R))$ ✗
Interchange σ and π	$\pi_A(\sigma_C(R)) = \sigma_C(\pi_A(R))$	$\pi_A(\sigma_C(R)) = \sigma_C(\pi_A(R))$
JOIN / $\times$	$\sigma_C(R \Join S) = \sigma_C(R) \Join S$	$\pi_A(R \Join S) \neq \pi_A(R) \Join \pi_A(S)$
Union ($\cup$)	$\sigma_C(R \cup S) = \sigma_C(R) \cup \sigma_C(S)$	$\pi_A(R \cup S) = \pi_A(R) \cup \pi_A(S)$
Intersection ($\cap$)	$\sigma_C(R \cap S) = \sigma_C(R) \cap S$ OR $\sigma_C(R \cap S) = R \cap \sigma_C(S)$	$\pi_A(R \cap S) \neq \pi_A(R) \cap \pi_A(S)$
Difference ($-$)	$\sigma_C(R - S) = \sigma_C(R) - S$	$\pi_A(R - S) \neq \pi_A(R) - \pi_A(S)$
Set operations (θ)	$\sigma_C(R \theta S) = \sigma_C(R) \theta S$ $\theta = \cup, \cap, -$	—
Cartesian Product ($\times$)	$\sigma(R \times S) = R \times S$	$\pi_A(R \times S) \neq \pi_A(R) \times \pi_A(S)$

← Memorize लायक ना

→ selection follows commutative as the order of row doesn't matter

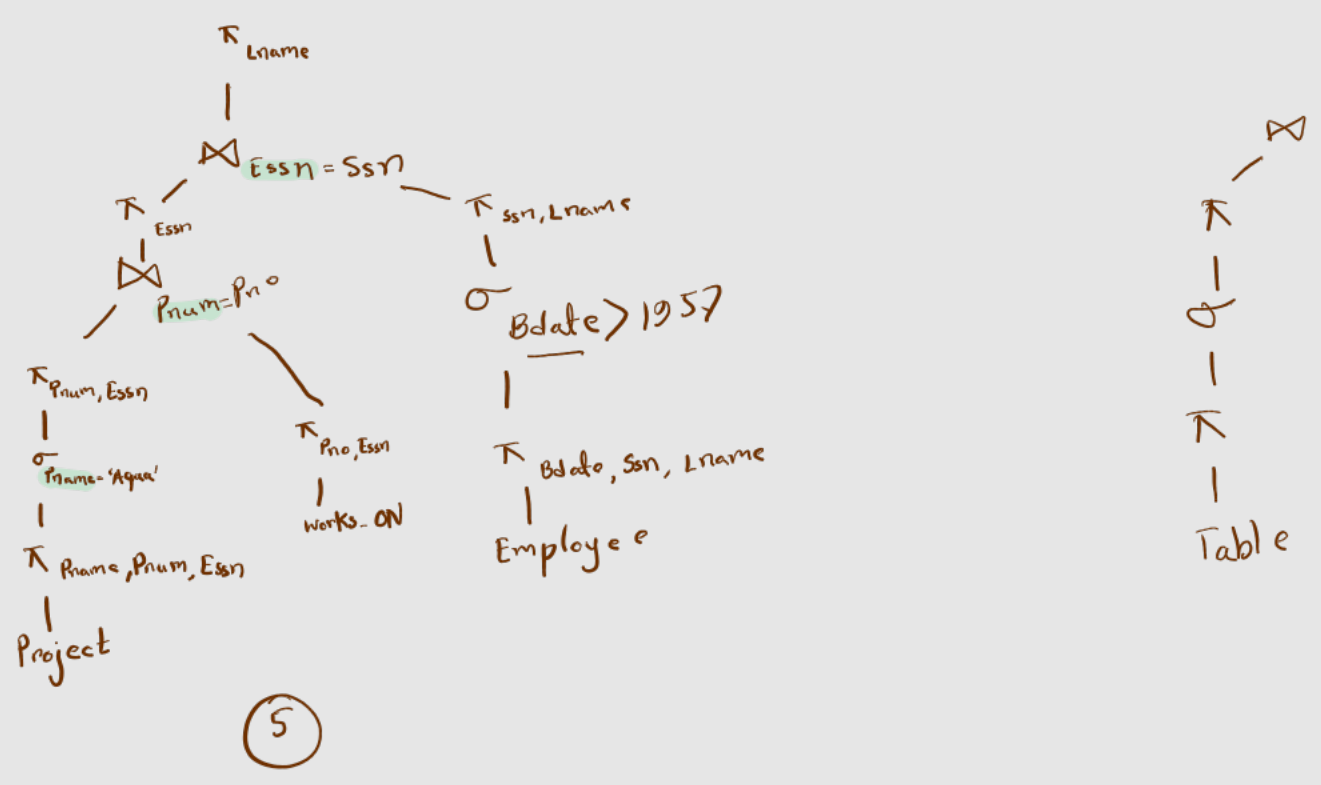
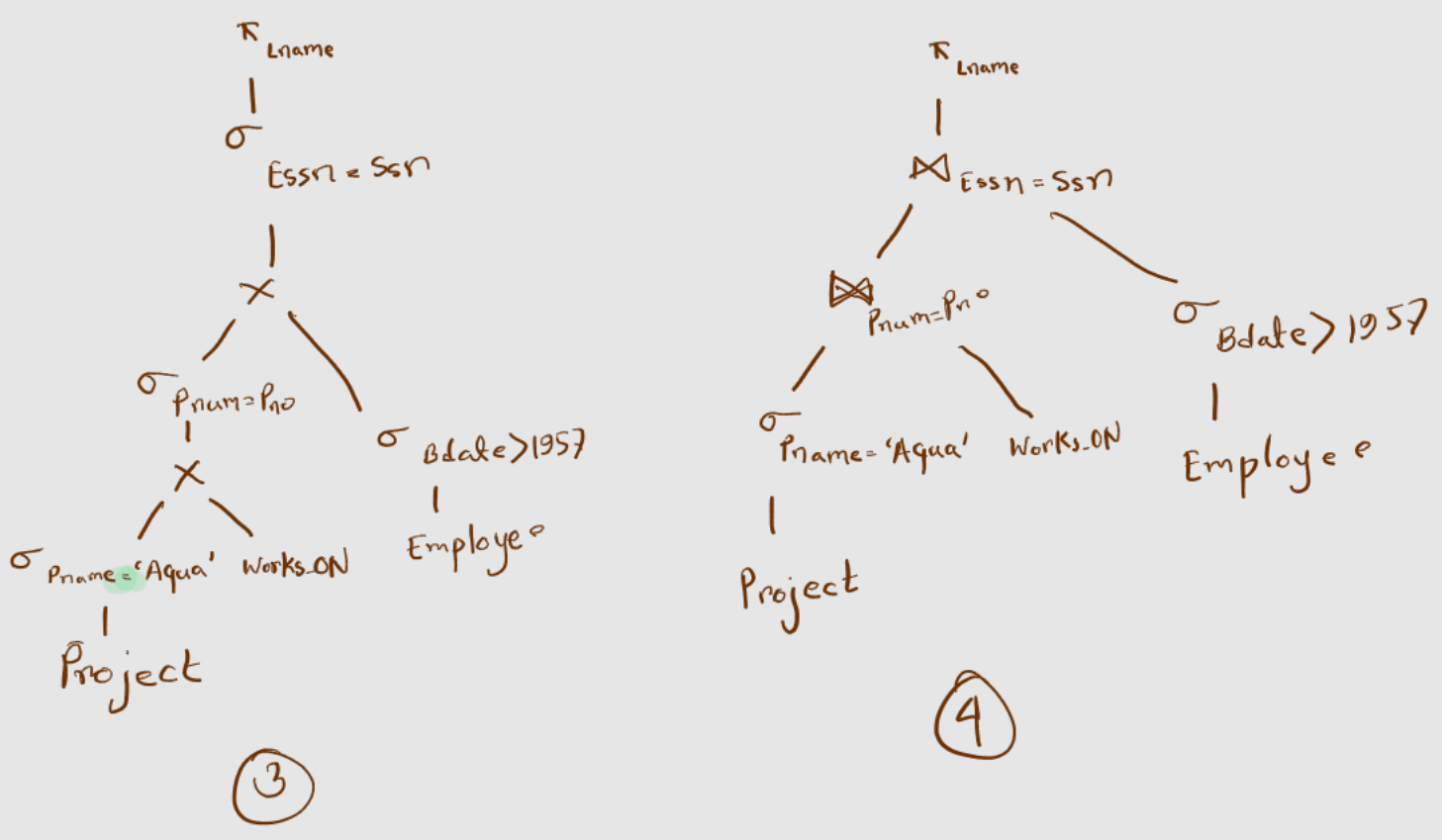
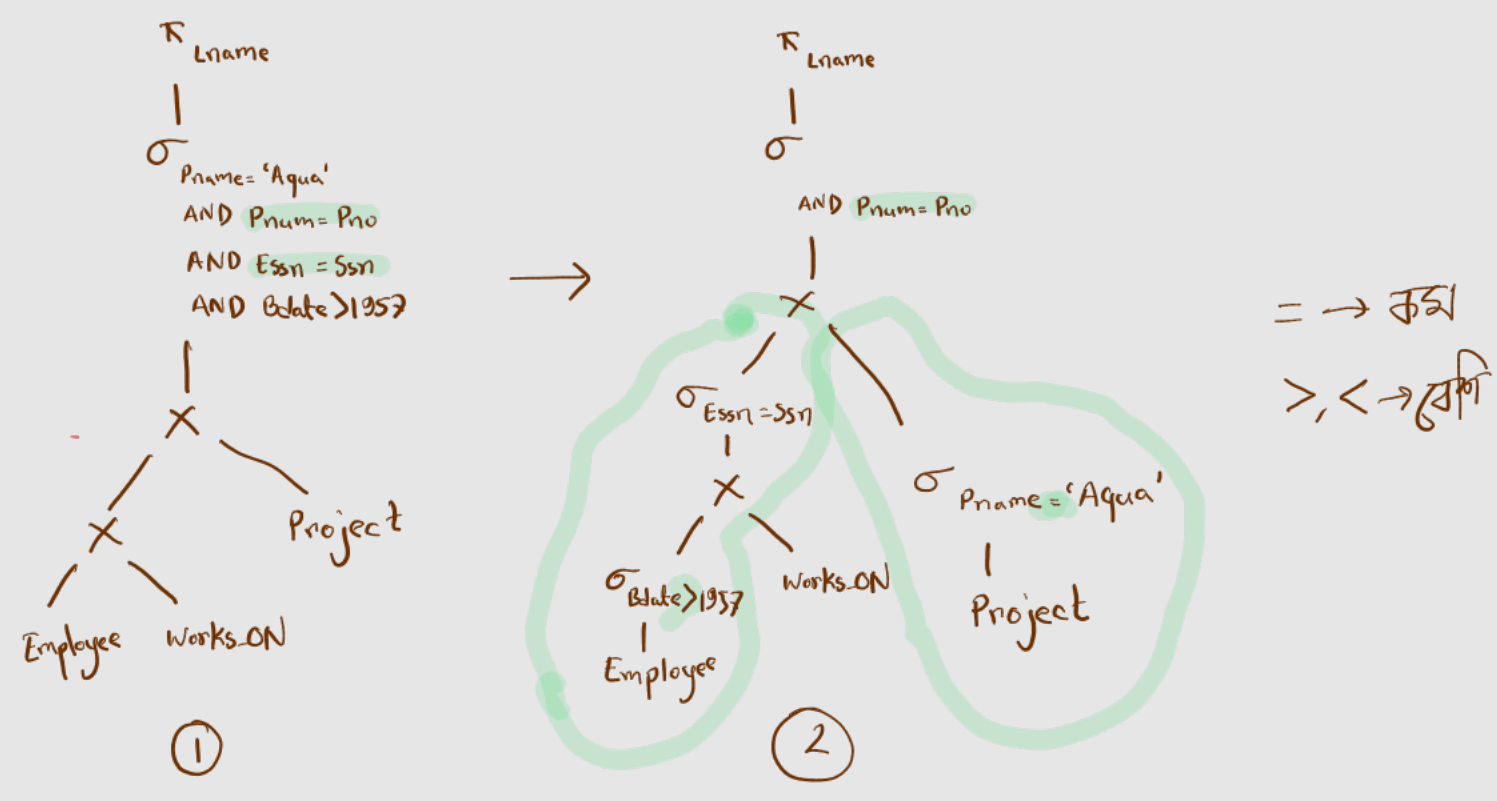
④ send Projection (π) below :

Selection Pushing Before join

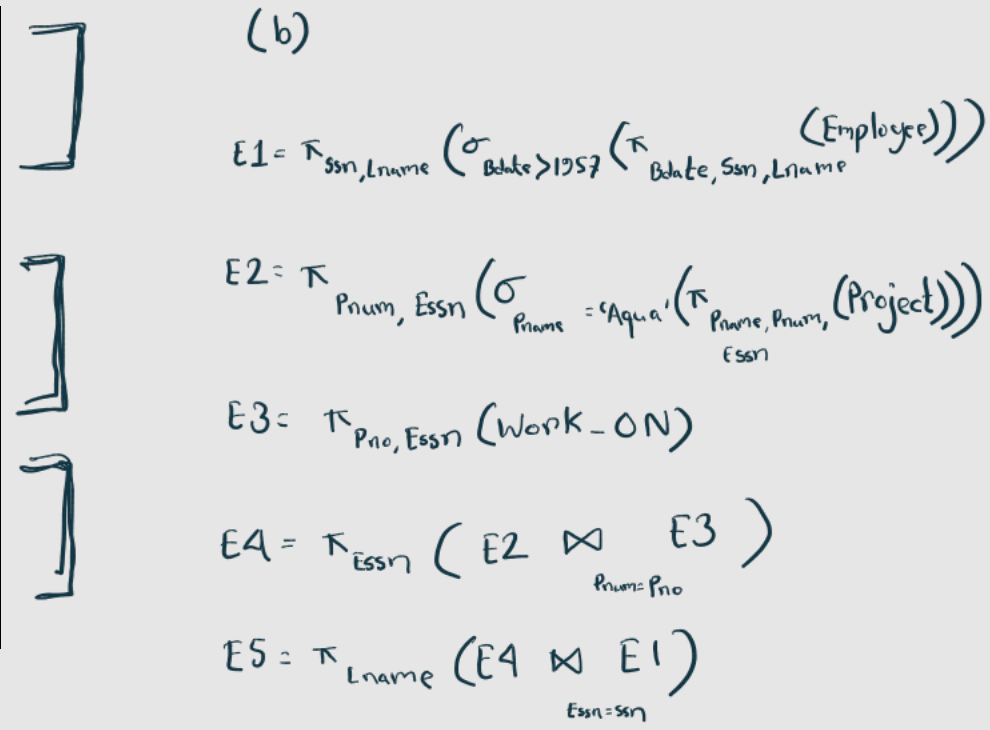


$\square$ SELECT Lname FROM Employee, Project, Works_On WHERE
 Pname = 'Aqua' AND Pnum = Pno AND Essn = Ssn AND Bdate > 1957

- (a) Find the modified SQL query
 * (b) Write RA expression for modified query tree



```
(a) SELECT e.Lname FROM (
      SELECT Ssn, Lname
      FROM Employee
      WHERE Bdate > 1957
    ) e
JOIN (
      SELECT w.Essn FROM (
        SELECT Pnumber
        FROM Project
        WHERE Pname = 'Aquarius'
      ) p
      JOIN (
        SELECT Essn, Pno
        FROM Works_ON
      ) w ON p.Pnumber = w.Pno
    ) pw ON pw.Essn = e.Ssn
```



Question 2:
Draw the query tree for the following SQL query then optimize the tree assuming the conditions of Question 1 and write down the relational algebra expression for your final query.
Select Dname, FName, LName
From Department join Project join Employee
On Department.Dnumber = Project.Dnum
and Depatment.Mgr_Ssn = Emplayee.Ssn
where Project.Location = 'BRAC University'
and Employee.Address = 'Gulshan 1, Dhaka'
and Department.Mgr_start_date < '01/01/2020'

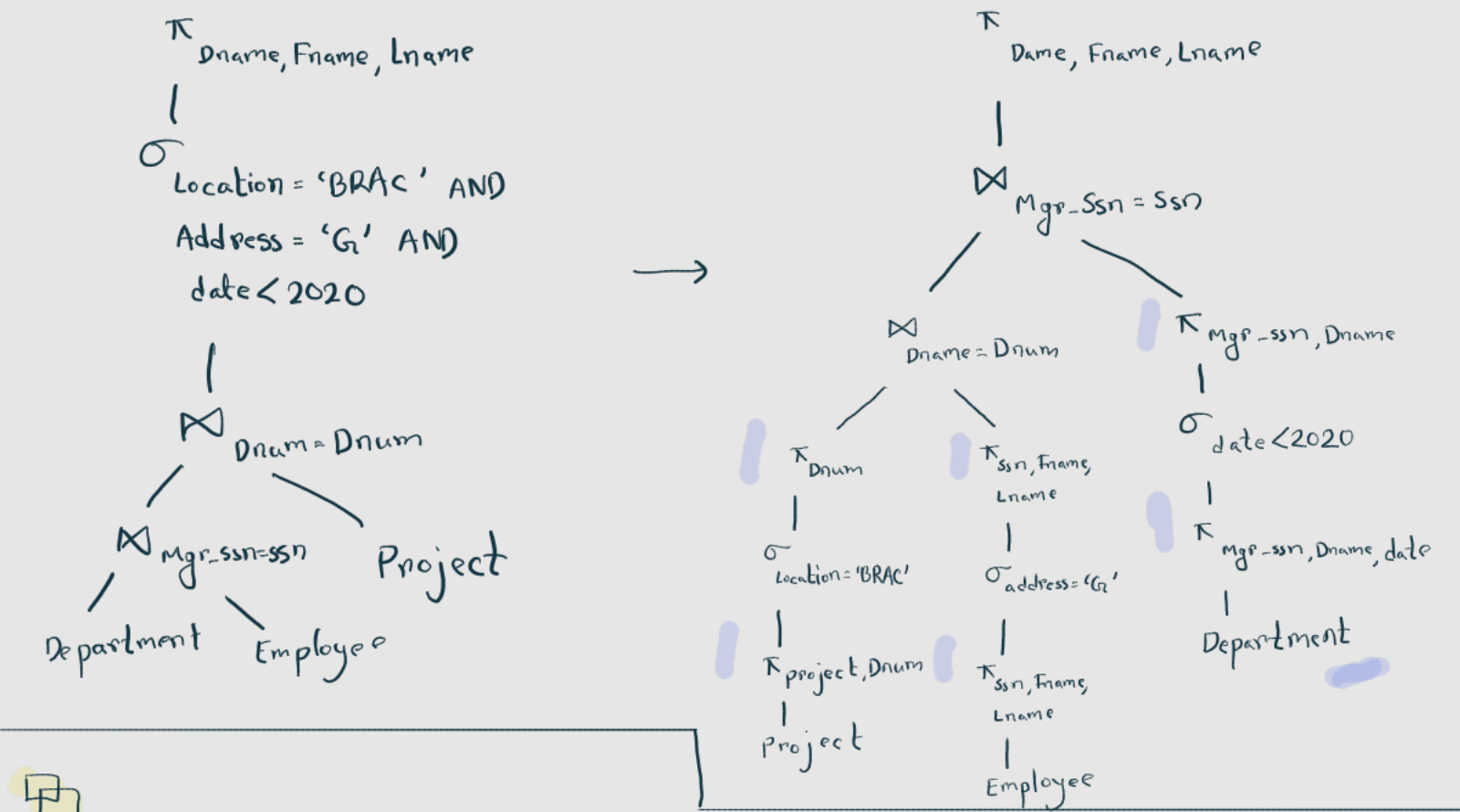
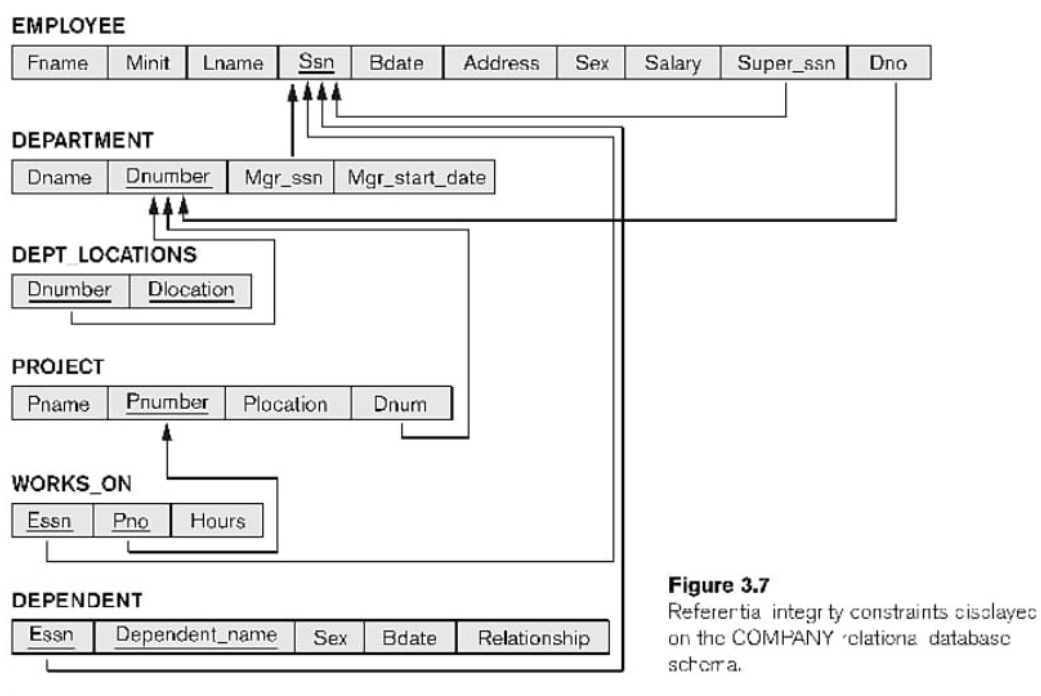


Table	attribute	row	condition
A	genre	1500	10% - pop 5% - rock
A	city	1500	25% - Tokyo 15% - Dhaka

Now,
 $C_1 = \sigma_{genre = 'pop'} (A)$
 $C_2 = \sigma_{city = 'Dhaka'} (A)$
which will be selected first?
show calculation.

Ans;
For C_1 ,
 $1500 \times \frac{10}{100} = 150$
For C_2 ,
 $1500 \times \frac{15}{100} = 225$
So, select C_1 earlier, it will filter out more rows.
RA Tree:
$$\sigma_{C_2} \mid \sigma_{C_1} \mid A$$

Assume the following is the relational schema for all questions in this category.

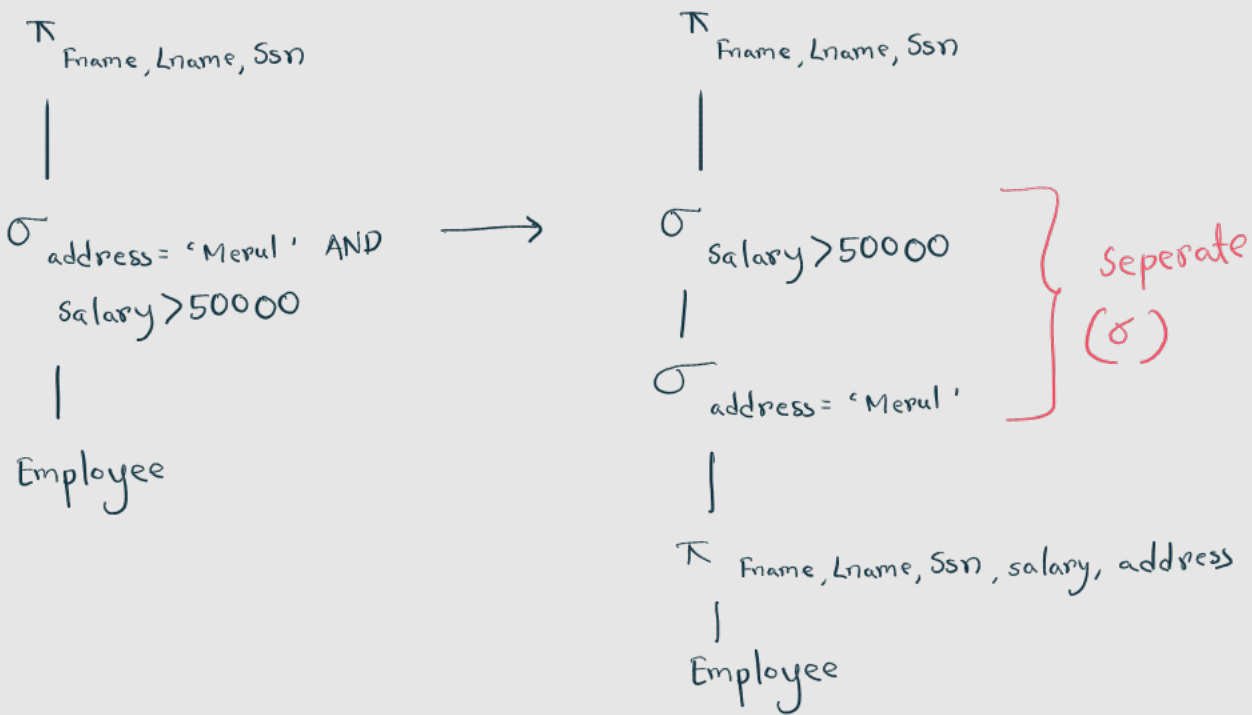


Question 1:

Draw the query tree for the following SQL query then optimize the tree assuming breaking select operations and pushing them downwards in the tree is better to filter records early and, in addition to that, projecting as early as possible is better to hold smaller records in memory during query execution. Then write the relational algebra expression for your modified query tree.

Select Fname, Lname, Ssn
from EMPLOYEE
where address = 'Merul Badda' and Salary > 50000

Ans:



Answer 1:

Skipping the tree drawing part. Here is the final query in relational algebra format.

E1 = $\pi_{Fname, Lname, Ssn, Address, Salary}$ (Employee)

E2 = $\sigma_{salary > 50000}$ ($\sigma_{address = 'Merul Badda'}$ E1)

E3 = $\pi_{Fname, Lname, Ssn}$ (E2)